

School of Computer Science and Statistics, Trinity College Dublin

yuyang.lin.hou@gmail.com +353 87 2746975

Personal Website: <https://laifsv1.github.io/yuyanglin.github.io>

<https://scholar.google.com/citations?user=m0eRUQYAAAAJ&hl=en>

<https://orcid.org/0000-0001-5783-9454>

I am a [Research Fellow](#) at Trinity College Dublin working in [formal methods](#), [programming languages](#), and [automated verification](#). My research develops semantics-based verification techniques and tools for higher-order open programs and program equivalence, with a recent focus on privacy and smart contracts.

EDUCATION

- PhD Computer Science – Queen Mary University of London [2022] ([Viva Passed: 2020¹](#))
- MSci Computer Science – University of Birmingham [2016] ([First Class](#))
- BSc Computer Science – University of Birmingham [2014] ([First Class](#))

WORK EXPERIENCE

2020/present – School of Computer Science and Statistics, Trinity College Dublin

- [Research Fellow](#) in the Software and Systems Discipline with a focus in Formal Methods
- **Funded Project Titles (Primary or Co-Author in Proposal):**
 - [Lero SFI](#): Core Technology for Enabling Regression Verification in Concurrent Software
 - Science Foundation Ireland grant [13/RC/2094_2](#)
 - [Cisco & Ethereum Foundation](#): Automated Relational Verification of Smart Contracts
 - Cisco University Research Program Fund
 - Ethereum Foundation grant [FY23-1127](#)
 - [Government of Ireland Postdoctoral Fellowship](#): Equivalence Verification for Privacy in Erlang
 - Irish Research Council project [GOIPD/2024/809](#)

2016/19 – School of Electronic Engineering and Computer Science, Queen Mary University of London

- [Demonstrator](#) for *Algorithms and Data Structures, Automata and Formal Languages, Functional Programming, Computer Systems and Networks, Introduction to Object-Oriented Programming, and Compilers*. **Responsibilities:** occasional lecturing, interacting with students in labs and lectures, grading assignments, adding and checking questions for lab assignments, automation of testing and grading of software assignments.
- [Organising committee](#) for the conference *Highlights of Logic, Games and Automata*, London 2017.

2015 – College of Engineering and Physical Sciences, University of Birmingham

- [Demonstrator](#) for *Foundations of Computer Science*. **Responsibilities:** interacting with students in labs, workshop lecture, creating and checking model answers for assignments.
- [Paid internship](#). Developed proof checker (<https://github.com/Laifsv1/YUP>).

2015 – King Edward VI Camp Hill School for Boys

- [Teacher assistant](#). Taught Year 8 and 12 students. **Responsibilities:** collaborated with the teacher and independently developed and delivered lectures.

2010 – SINGRAF: *Engineering for the Advertising Industry*

- Worked with CAD/CAM software and machines, and managed spreadsheets.

SKILLS

- **Main Programming Languages:** OCaml, Java, Python, Haskell
- **Verification and Proof Languages:** Agda, PROMELA, K, SMT-LIB (Z3)
- **Other Programming Languages:** C/C++, MATLAB, CUDA, Racket (Lisp), Solidity, EVM Yul
- **Natural Languages:** English, Spanish; spoken Mandarin

¹ Graduation Ceremony and Award delayed due to COVID19

PUBLICATIONS AND SCHOLARLY WORK

- **Publications (refereed)**
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *Fully Abstract Normal Form Bisimulation for Call-by-Value PCF*. **JACM**, 2025. <https://doi.org/10.1145/3765737> (*Invited for publication*)
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *An Operational Semantics for Yul*. **SEFM**, 2024. https://doi.org/10.1007/978-3-031-77382-2_19
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *Pushdown Normal-Form Bisimulation: A Nominal Context-Free Approach to Program Equivalence*. **LICS**, 2024. <https://doi.org/10.1145/3661814.3662103>
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *Fully Abstract Normal Form Bisimulation for Call-by-Value PCF*. **LICS**, 2023. <https://doi.org/10.1109/LICS56636.2023.10175778> (*Distinguished Paper Award*)
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *From Bounded Checking to Verification of Equivalence via Symbolic Up-to Techniques*. **TACAS**, 2022. https://doi.org/10.1007/978-3-030-99527-0_10
 - Lin, Y.-Y. and Tzevelekos, N. *Symbolic Execution Game Semantics*. **FSCD**, 2020. <https://doi.org/10.4230/LIPICs.FSCD.2020.27>
 - Lin, Y.-Y. and Tzevelekos, N. *A Bounded Model Checking Technique for Higher-Order Programs*. **SETTA**, 2019. https://doi.org/10.1007/978-3-030-35540-1_1
- **Selected Technical Reports**
 - Koutavas, V., Lin, Y.-Y., and Tzevelekos, N. *A Bounded Game Semantics Checker for Precise Smart Contract Analysis*. **CoRR**, 2026. <https://arxiv.org/abs/2512.22417v2>
- **Theses**
 - **PhD**: “Bounded Verification of Higher-Order Stateful Programs”, Queen Mary University of London, School of Electronic Engineering and Computer Science, 2021.
 - **MSci**: “Compiler Optimisations for High-Level Synthesis”, University of Birmingham, College of Engineering and Natural Sciences, 2016.
- **Selected Presentations**
 - **SEFM 2024**: *An Operational Semantics for Yul*
 - **LICS 2024**: *Pushdown Normal-Form Bisimulation: A Nominal Context-Free Approach to Program Equivalence*
 - **TACAS 2022 (ETAPS)**: *From Bounded Checking to Verification of Equivalence via Symbolic Up-to Techniques*
 - **FSCD 2020**: *Symbolic Execution Game Semantics*
 - **IMDEA Invited Talks 2019**: *A Bounded Model Checking Technique for Higher-Order Programs*

RESEARCH SOFTWARE AND ARTEFACTS

- **YulTracer v0.2** (2026): game-semantics-based bounded safety checker for Yul.
 - <https://doi.org/10.5281/zenodo.18280182>; <https://doi.org/10.5281/zenodo.17102811>
- **YulTracer v0.1** (2024): modular interpreter and future symbolic-execution tool for Yul, based on the operational semantics for Yul.
 - <https://doi.org/10.5281/zenodo.12511493>; <https://doi.org/10.5281/zenodo.12588075>
- **Hobbit-PDNF** (2024): equivalence verification tool for open higher-order programs with local state.
 - <https://doi.org/10.5281/zenodo.11128049>
- **pcfeq** (2023): equivalence verification tool for call-by-value PCF.
 - <https://doi.org/10.5281/zenodo.7864539>
- **Hobbit** (2021): equivalence verification tool for open higher-order programs with local state.
 - <https://doi.org/10.5281/zenodo.7864658>
- **HOLiK** (2019): safety verification tool for open higher-order programs with global state.
 - <https://github.com/LaifsV1/HOLiK>
- **BMC2** (2018): safety verification tool for closed higher-order programs with global state.
 - <https://github.com/LaifsV1/BMC-2>
- **YUP** (2015): proof checker for simple program-correctness proofs of functional programs.
 - <https://github.com/LaifsV1/YUP>